INFOSYS SPRINGBOARD INTERNSHIP 6.0

OpenDeepResearcher

Agentic LLM Research Framework

Full Project Documentation — All Milestones

Submitted By	Surapaneni Jaya Ravi Seshank
Program	Infosys Springboard Internship 6.0
Mentor	Deepanshu Choudhary
Submission Date	April 2026

1. Project Overview

1.1 Description

OpenDeepResearcher is an AI-powered research assistant designed to autonomously conduct in-depth explorations of complex topics. Traditional research requires hours of manually searching, reading, and summarizing information from multiple sources. This project solves that problem by building an intelligent system that does all of that automatically.

The system leverages the power of Large Language Models (LLMs), agentic workflows, and web search APIs to plan, retrieve, analyze, and summarize information from credible sources on the internet. By simulating the behavior of a skilled human researcher, OpenDeepResearcher enables users to generate high-quality, multi-perspective research reports efficiently and cost-effectively.

This project is built entirely using open-source tools and runs locally on the development machine, making it privacy-friendly and cost-effective without relying on paid cloud APIs for model inference.

1.2 Problem Statement

Manual research is time-consuming, inconsistent, and difficult to scale. A researcher must manually identify search queries, evaluate credibility, read dozens of sources, synthesize findings, and write a coherent report — a process that can take hours or days for a single topic.

OpenDeepResearcher addresses this by automating the entire pipeline using an agentic multi-step workflow where specialized AI agents each handle a distinct responsibility: planning, searching, and writing.

1.3 Project Objectives

- Build an autonomous AI research assistant capable of end-to-end research without human intervention.
- Use local LLMs via LM Studio to avoid dependency on paid cloud APIs.
- Integrate real-time web search using the Tavily API to ground responses in current information.
- Design a modular multi-agent architecture using LangGraph for maintainability and scalability.
- Implement session memory so users can maintain context across multiple research queries.
- Generate structured, citable research reports with source attribution and timestamps.
- Build an interactive web UI using Streamlit with a ChatGPT-like experience.

2. Technology Stack

The project uses a carefully selected stack focused on local inference, agentic orchestration, and real-time web retrieval.

2.1 Technology Summary Table

Category	Technology	Purpose
Language	Python 3.10+	Core language for all agent logic and pipeline control.
Agentic Framework	LangGraph	Multi-agent workflow orchestration using directed state graphs.
LLM Library	LangChain	LLM integration, message handling, and agent coordination.
Local LLM Runtime	LM Studio	Runs Qwen2.5-3B-Instruct locally with OpenAI-compatible API.
Language Model	Qwen2.5-3B-Instruct	Instruction-tuned LLM for planning and writing tasks.
Web Search	Tavily API	AI-optimized real-time web search with structured results.
Memory	MemorySaver (LangGraph)	Persistent session memory with thread-based continuity.
UI Framework	Streamlit	Interactive chat-based web interface.
Environment	python-dotenv, venv	Dependency isolation and secure API key management.
Version Control	Git / GitHub	Source code management and project submission.

2.2 Technology Decisions and Rationale

Python 3.10+

Python was selected as the core language due to its rich ecosystem of AI and machine learning libraries, clean syntax for rapid prototyping, and strong community support. The typing module (TypedDict, Annotated) is used extensively for type-safe state management in the LangGraph pipeline.

LangGraph

LangGraph is used to define and manage the multi-agent workflow as a directed state graph. Each agent (Planner, Searcher, Writer) is a node, and the graph controls the execution order and state passing between them. LangGraph's built-in MemorySaver checkpointing enables thread-level memory persistence.

LM Studio with Qwen2.5-3B-Instruct

LM Studio provides a local inference runtime with an OpenAI-compatible REST API, eliminating the need for paid cloud model providers. The Qwen2.5-7B-Instruct model was originally planned but was replaced with the 3B version due to hardware memory constraints on the development machine. This change was discussed with and approved by the project mentor. The 3B model delivers adequate output quality for the current project scope.

Tavily API

Tavily is a search API specifically designed for AI agents. Unlike generic search engines, it returns structured results with titles, URLs, and content summaries in a format that is directly consumable by LLMs. This significantly reduces preprocessing and improves the quality of information passed to the Writer Agent.

Streamlit

Streamlit provides a fast way to build interactive web applications in pure Python. It was chosen for the UI layer because it integrates naturally with the Python backend pipeline, supports session state management, and allows HTML/CSS injection for custom styling.

3. System Architecture

3.1 Architectural Overview

The system follows a modular multi-agent architecture built on the Single Responsibility Principle. Each agent has a clearly defined role and communicates through a shared typed state object managed by LangGraph. The pipeline processes each research query through three sequential stages with an optional reflection loop for quality improvement.

3.2 Pipeline Flow

1. User enters a research topic through the Streamlit chat interface.
2. Planner Agent receives the topic and breaks it into 5 focused, distinct web search queries.
3. Searcher Agent queries the Tavily API for each sub-question and collects real web results.
4. Writer Agent synthesizes all retrieved information into a structured, cited summary.
5. Report Generator compiles the full research into a downloadable Markdown or text report.
6. Session Memory stores the exchange for contextual continuity in subsequent turns.

3.3 Architecture Flow Diagram (Text Representation)

```
User Query
     ↓
[Planner Agent]  — Generates 5 search queries using LLM
     ↓
[Searcher Agent] — Queries Tavily API, collects real web results
     ↓
[Writer Agent]   — Synthesizes summary with [1][2] citations
     ↓
[Report Generator] — Creates structured .md report with timestamp
     ↓
Final Report delivered to user + stored in MemorySaver
     ↑
Reflection Loop — Memory context fed back into next query
```

3.4 LangGraph State Object

All agents share a typed state object defined using Python's TypedDict. This ensures type safety and prevents data corruption between pipeline stages.

```
class ResearchState(TypedDict):
    messages:  Annotated[List, operator.add]  # full conversation history
    topic:     str                             # current research topic
    plan:      List[str]                       # queries from Planner
    results:   List[dict]                      # raw results from Tavily
    summary:   str                             # written summary
    report:    str                             # full structured report
    turn:      int                             # conversation turn number
```

3.5 LangGraph Execution Graph

The directed acyclic graph (DAG) ensures deterministic transitions between agents. MemorySaver is attached as a checkpointer so state is preserved between invocations, enabling multi-turn research continuity.

```
graph = StateGraph(ResearchState)
graph.add_node('planner',  planner_node)
graph.add_node('searcher', searcher_node)
graph.add_node('writer',   writer_node)
graph.add_edge(START,      'planner')
graph.add_edge('planner',  'searcher')
graph.add_edge('searcher', 'writer')
graph.add_edge('writer',    END)
app = graph.compile(checkpointer=MemorySaver())
```

4. Project Structure

4.1 Directory Layout

```
OpenDeepResearcher/
├── app.py                  ← Streamlit web application (all milestones)
├── main.py                 ← CLI runner for terminal-based research
├── .env                    ← API keys and model config (not committed to Git)
├── chats.json              ← Persisted chat threads (auto-generated)
├── requirements.txt        ← Python dependency list
├── README.md               ← Project documentation
└── opensearch/             ← Agent module package
    ├── __init__.py         ← Package exports
    ├── planner.py          ← Planner Agent
    ├── searcher.py         ← Searcher Agent
    └── writer.py           ← Writer Agent + Report Generator
```

4.2 Environment Configuration (.env)

All sensitive credentials are stored in a .env file and loaded at runtime using python-dotenv. This ensures that API keys are never hardcoded in source files and are excluded from version control via .gitignore.

```
TAVILY_API_KEY=tvly-dev-xxxxxxxxxxxxxxxxxxx
LM_STUDIO_BASE_URL=http://127.0.0.1:1234
LM_STUDIO_MODEL=qwen2.5-3b-instruct
```

5. Agent Components

5.1 Planner Agent (planner.py)

The Planner Agent is the entry point of the research pipeline. It receives the user's research topic and uses the LLM to decompose it into exactly 5 focused, distinct search queries. These queries act as a structured research roadmap, ensuring the system explores the topic from multiple angles rather than producing a single generic answer.

Why LLM is Used for Planning

Planning requires semantic understanding and reasoning — the ability to identify what sub-questions would yield the most useful information about a topic. This is inherently a language task that benefits from LLM reasoning capability. Rule-based decomposition would produce rigid, low-quality queries.

Context-Awareness

In multi-turn sessions, the Planner receives the last 6 messages from the conversation history as context. This means that if a user asks a follow-up question, the Planner generates contextually-aware queries that build on previous research rather than treating each question in isolation.

Implementation

```
def plan(self, topic: str, history_context: str = '') -> list[str]:
    ctx = f'Conversation so far:\n{history_context}\n' if history_context else ''
    prompt = (
        f'You are a research planner.{ctx}\n'
        f'Break this topic into exactly 5 short, distinct web search queries.\n'
        f'Return ONLY a numbered list. No extra text.\n\n'
        f'Topic: {topic}'
    )
    resp = self.llm.invoke([HumanMessage(content=prompt)])
    return [clean_line(l) for l in resp.content.split('\n')][:5]
```

5.2 Searcher Agent (searcher.py)

The Searcher Agent is responsible for information retrieval. It takes each query generated by the Planner and submits it to the Tavily API, collecting structured web search results including title, URL, and content for each source. The Searcher does not summarize — it returns raw, full content so the Writer Agent has maximum context for synthesis.

Why Searcher Does Not Summarize

Summarizing at the retrieval stage would remove critical contextual details that the Writer Agent needs to produce a high-quality, accurate response. Passing full content reduces hallucination risk because the Writer is grounded in real source material rather than compressed abstractions.

Bug Fixed

A critical bug was identified in the original code: `os.getenv("tvly-dev-...")` was passing the API key itself as the variable name instead of `os.getenv("TAVILY_API_KEY")`. This caused the Tavily client to initialize with None, silently failing all searches.

Implementation

```
def search_many(self, queries: list[str]) -> list[dict]:
    all_results = []
    for q in queries:
        resp = self.client.search(query=q, max_results=3)
        all_results.extend(resp.get('results', []))
    return all_results
```

5.3 Writer Agent (writer.py)

The Writer Agent is the final stage of the synthesis pipeline. It receives the research topic, all raw search results collected by the Searcher, and the conversation history context. It uses the LLM to produce a detailed, well-structured 3-4 paragraph summary with inline citations in the format [1], [2] referencing the source materials.

Why Both Topic and Results Are Required

The research topic provides semantic context about what the summary should focus on. The raw search results provide factual grounding. Without the topic, the Writer may drift into tangential content. Without the raw results, the Writer would hallucinate. Both inputs together produce high-quality, accurate, on-topic summaries.

Milestone 4 Extension — Full Report Generation

In Milestone 4, the Writer was extended with a generate_full_report() method that produces a structured research document including a research plan section, the full summary with citations, a numbered source list with URLs, and a generation timestamp. This report can be downloaded directly from the UI.

```
def generate_full_report(self, topic, queries, results, summary) -> str:
    timestamp = datetime.now().strftime('%B %d, %Y at %H:%M')
    # Builds structured Markdown with sections:
    # - Research Plan (numbered queries)
    # - Summary of Findings (full paragraphs)
    # - Sources (numbered with URLs)
    # - Timestamp footer
```

6. Memory System

6.1 Overview

The memory system is one of the most important features of OpenDeepResearcher. It enables the research agent to maintain context across multiple turns within a conversation thread, so each new question builds on the previous research rather than starting from scratch.

6.2 MemorySaver

LangGraph's MemorySaver acts as a checkpointing backend for the state graph. When the graph is invoked with a thread_id in the config, MemorySaver automatically saves and restores the graph state between invocations. This means the full message history, research results, and turn counter are all preserved without any manual serialization.

```
config = {'configurable': {'thread_id': thread_id}}
result = graph.invoke(initial_state, config=config)
```

Each unique thread_id corresponds to an independent research session. Multiple threads can coexist in memory simultaneously, allowing users to maintain several research contexts in parallel.

6.3 Thread Tracking

Each thread in the system has the following tracked properties:

Property	Description
thread_id	Unique 8-character identifier generated with uuid.uuid4().
title	Auto-set from the first message in the thread (first 38 characters).
created	Timestamp when the thread was created (HH:MM format).
turn	Integer counter tracking how many research exchanges have occurred.
messages	Full list of user and assistant messages with all research data.

6.4 Session Persistence (chats.json)

Thread data is persisted to a local chats.json file after every research turn. This means that if the Streamlit server is restarted, all previous threads and their full conversation history are automatically restored from disk on the next launch. This provides durable session memory beyond the in-memory MemorySaver.

```
def save_chats(threads: dict):
    with open('chats.json', 'w') as f:
        json.dump(serializable_threads, f, indent=2)
```

6.5 Context Injection

When a new query arrives in an existing thread, the last 6 messages from the conversation history are formatted and injected into both the Planner and Writer prompts. This gives the agents awareness of what topics have already been researched, enabling genuinely contextual follow-up questions and preventing redundant searches.

For example, if the user first asks about FPGA architecture and then asks a follow-up about FPGA applications, the Planner will generate queries that complement the earlier research rather than duplicating it.

6.6 Thread Limits

Each thread is limited to a maximum of 10 research turns (MAX_TURNS = 10). Beyond this limit, a warning is displayed and the user is encouraged to start a new thread. This prevents context window overflow in the LLM prompts, which could degrade response quality as the history grows very long.

7. User Interface (Streamlit App)

7.1 Overview

The Streamlit web application provides a full ChatGPT-like research interface. It is implemented in app.py and covers all four project milestones in a single file. The UI uses a dark agent-style aesthetic with IBM Plex Sans and IBM Plex Mono fonts, a sidebar for thread management, and a main chat area for research interactions.

7.2 UI Components

Sidebar

- Agent header with version badge (agent v0.3 · langgraph)
- Memory status badge showing total threads and turns stored
- New Thread button to start a fresh research session
- Thread list showing all past sessions with turn count and creation time
- Open and Delete buttons for each thread
- Status indicators for LM Studio connection, Tavily API, and MemorySaver

Top Bar

- Current thread ID displayed as a badge
- Turn counter (current / MAX_TURNS)
- Active model name from environment variable
- Current date

Main Chat Area

- Welcome screen with pulsing orb animation and 4 suggestion chips
- User messages with U avatar and turn number
- Agent responses with research plan, full summary, source chips, and turn/timestamp
- Memory context tag on responses that use prior conversation history
- Download .md and Download .txt buttons on every response (Milestone 4)
- Memory limit warning when thread reaches 10 turns

Input Bar

- Styled text input with accent-colored focus border
- Run button to submit queries
- Live memory indicator showing active dot, turn count, and thread ID
- Disclaimer footer

7.3 Thinking Indicator

While the pipeline is processing, a three-step animated thinking indicator is displayed in the chat area. It shows the current pipeline stage (planning queries / searching web / writing report) with a progress bar that fills from 0 to 100% as each stage completes. This gives the user real-time feedback on processing status.

7.4 Report Download (Milestone 4)

Every research response includes two download buttons that allow the user to save the full structured research report:

- Download .md — saves as a formatted Markdown file with headers, sections, and source list
- Download .txt — saves as plain text for maximum compatibility

Each report is named report_{turn_number}.md and includes the research plan, full summary, numbered source list with URLs, and a generation timestamp.

8. Project Milestones

8.1 Milestone Overview Table

Milestone	Weeks	Focus	Status
1	Weeks 1–2	Foundation Setup	
2	Weeks 3–4	Core Agent Development	
3	Weeks 5–6	UI and Memory Integration	
4	Weeks 7–8	Refinement and Final Output	

8.2 Milestone 1 — Foundation Setup (Weeks 1–2)

The first milestone established the complete technical foundation of the project. Every subsequent milestone builds directly on the work done here.

Objectives

- Set up the Python development environment with virtual environment and dependency management.
- Design and document the multi-agent system architecture.
- Integrate a local LLM via LM Studio with an OpenAI-compatible API interface.
- Configure the Tavily API for web search access.
- Validate the complete pipeline with a basic stub research flow.

Environment Setup

A Python virtual environment was created to isolate all project dependencies. This ensures reproducibility and prevents version conflicts with other Python projects on the same machine.

```
python -m venv venv
venv\Scripts\activate           # Windows
source venv/bin/activate          # macOS/Linux
pip install -r requirements.txt
```

LM Studio Integration

LM Studio was installed and configured to serve the Qwen2.5-3B-Instruct model locally. The server runs on http://localhost:1234/v1 and provides a fully OpenAI-compatible REST API. The model was originally planned as Qwen2.5-7B-Instruct but was downgraded to the 3B version due to RAM and GPU memory constraints on the development machine. This was approved by the project mentor.

```
llm = ChatOpenAI(
    base_url='http://127.0.0.1:1234/v1',
    api_key='lm-studio',
    model='qwen2.5-3b-instruct',
)
```

Issue Encountered and Resolved

During Tavily API setup, a MissingAPIKeyError was encountered. Investigation revealed that the .env file was not being loaded before the TavilyClient was initialized. The fix was to call load_dotenv() at the top of the script, before any os.getenv() calls, and to verify the exact variable name used in the .env file.

Milestone 1 Evaluation Results

- Python virtual environment fully configured and reproducible.
- Project architecture defined with clear agent responsibilities and data flow.
- LM Studio server running and returning valid LLM responses.
- Tavily API returning structured web search results.
- Stub research flow executed end-to-end producing coherent output.

8.3 Milestone 2 — Core Agent Development (Weeks 3–4)

The second milestone transformed the stub pipeline into a fully functional multi-agent research system. Each agent was implemented as a proper class with a well-defined interface, and the full LangGraph execution graph was built and validated.

Planner Agent Implementation

The Planner was implemented as a class with a plan() method that takes a research topic and returns a list of 5 focused search queries. Prompt engineering was used to ensure the LLM returns only the numbered list without additional commentary, making parsing reliable.

Searcher Agent Implementation

The Searcher was implemented as a class wrapping the TavilyClient. It provides both a single-query search() method and a batch search_many() method that iterates over all planner queries. Error handling was added to gracefully handle network failures and missing API keys without crashing the pipeline.

Writer Agent Implementation

The Writer was implemented as a class with a summarize() method. It receives the topic, raw search results, and optional conversation history, then uses the LLM to write a structured 3-4 paragraph summary with inline source citations.

LangGraph Pipeline

The three agents were connected into a LangGraph StateGraph with deterministic transitions: START → planner → searcher → writer → END. The shared ResearchState TypedDict passes data between nodes cleanly without global variables.

Milestone 2 Evaluation Results

- All three agents implemented and unit-tested independently.
- LangGraph pipeline executes successfully end-to-end.
- Planner generates relevant, distinct search queries.
- Searcher retrieves real web content via Tavily.
- Writer produces coherent, factually-grounded summaries.

8.4 Milestone 3 — UI and Memory Integration (Weeks 5–6)

The third milestone added the interactive Streamlit web interface and the full session memory system, transforming the CLI tool into a production-quality web application.

Streamlit UI

A complete ChatGPT-like interface was built using Streamlit with custom CSS styling. The UI features a dark agent aesthetic with the IBM Plex font family, animated suggestion chips on the welcome screen, a thread management sidebar, and real-time thinking indicators during research.

Session Memory with MemorySaver

LangGraph's MemorySaver was integrated as the graph checkpointer. Each research thread uses a unique thread_id as the memory key, ensuring that state is stored and restored correctly between invocations. This provides genuine multi-turn contextual research capability.

Thread Management

The UI supports multiple simultaneous research threads. Users can create new threads, switch between them, and delete old ones. Thread titles are auto-generated from the first message, and each thread tracks its turn count against the 10-turn maximum.

Chat Persistence

All thread data is serialized to chats.json after each turn, providing persistence across Streamlit server restarts. On startup, the app loads existing threads from disk so users can resume previous research sessions.

Milestone 3 Evaluation Results

- Streamlit UI integrated and fully interactive.
- Session memory functional with thread-based continuity.
- Multi-thread support working with correct state isolation.
- Chat history persisted to disk and restored on restart.
- Context injection confirmed to improve follow-up query quality.

8.5 Milestone 4 — Refinement and Final Output (Weeks 7–8)

The final milestone focused on output quality, structured report generation, and polishing the complete system for demonstration.

Structured Report Generation

The Writer Agent was extended with a generate_full_report() method that produces a complete structured research document. The report includes a title and generation timestamp, a numbered research plan showing all search queries used, the full multi-paragraph summary with inline citations, and a numbered source list with titles and URLs.

Download Functionality

Every research response in the UI now displays two download buttons: one for Markdown format (.md) and one for plain text (.txt). Files are named with the turn number for easy identification and organization.

Citation Format

The Writer Agent was updated to use numbered citation references [1], [2], [3] within the summary text, with a matching numbered source list at the end of the report. This mirrors the citation style used in academic and professional research documents.

Prompt Tuning

All three agent prompts were refined based on observed output quality. The Planner was instructed to generate queries that are short, distinct, and specific rather than verbose paraphrases of the topic. The Writer was instructed to be specific, cite source numbers, and explicitly connect findings to prior topics in multi-turn sessions.

Milestone 4 Evaluation Results

- Full structured report generated with sections, citations, and timestamps.
- Report download working for both Markdown and plain text formats.
- Inline citations [1][2] correctly map to source list entries.
- System behavior stable and reliable across multiple test topics.
- Documentation completed and demo prepared.

9. Installation and Setup Guide

9.1 Prerequisites

- Python 3.10 or above installed on the system
- LM Studio downloaded and installed from https://lmstudio.ai
- Qwen2.5-3B-Instruct (or 7B if hardware permits) downloaded inside LM Studio
- A Tavily API account and key from https://tavily.com
- Git installed for version control
- Stable internet connection for Tavily web searches

9.2 Step-by-Step Setup

7. Clone or download the project repository to your local machine.
8. Create a Python virtual environment in the project root:

```
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate      # macOS/Linux
```

9. Install all required dependencies:

```
pip install -r requirements.txt
```

10. Create a .env file in the project root with your credentials:

```
TAVILY_API_KEY=your_tavily_key_here
LM_STUDIO_BASE_URL=http://127.0.0.1:1234
LM_STUDIO_MODEL=qwen2.5-3b-instruct
```

11. Open LM Studio, load the Qwen2.5-3B-Instruct model, and start the local server.
12. Run the Streamlit application:

```
streamlit run app.py
```

13. The app will open automatically in your browser at http://localhost:8501

9.3 Running the CLI Version

For terminal-based research without the UI, the main.py CLI runner can be used directly:

```
python main.py
```

It will prompt for a research topic, run the full Planner-Searcher-Writer pipeline, and save the structured report to report.md in the project directory.

9.4 Verifying the Setup

To confirm everything is working before running the full app:

14. Verify LM Studio is running: open http://127.0.0.1:1234/v1/models in a browser — it should return a JSON list of available models.
15. Verify Tavily connection: ensure your TAVILY_API_KEY is valid and has remaining credits.
16. Run a quick test in Python:

```
from opensearch.searcher import Searcher
s = Searcher()
print(s.search('AI in healthcare'))
```

10. Challenges and Resolutions

Challenge	Root Cause	Resolution
LLM could not run smoothly on development machine.	Qwen2.5-7B-Instruct requires more RAM and GPU VRAM than available.	Switched to Qwen2.5-3B-Instruct after mentor approval. 3B performs adequately for this project scope.
MissingAPIKeyError from Tavily on first run.	load_dotenv() was not called before os.getenv(), so the environment variable was None.	Moved load_dotenv() to the top of the file, before all API client initialization.
Tavily search silently returning no results.	os.getenv() was called with the actual API key string instead of the variable name 'TAVILY_API_KEY'.	Fixed the bug in searcher.py: os.getenv('TAVILY_API_KEY').
LangGraph MemorySaver resetting between Streamlit reruns.	LangGraph graph was being rebuilt on every Streamlit interaction, creating a new MemorySaver each time.	Wrapped get_memory() and get_agents() with @st.cache_resource to persist them across reruns.
Thread history not surviving Streamlit server restarts.	st.session_state is ephemeral and does not persist to disk.	Implemented chats.json serialization: threads are saved after every turn and loaded on startup.
Planner generating redundant queries repeating the full topic phrase.	Insufficient prompt specificity — LLM appended 'overview', 'latest trends' etc. to the full topic string.	Refined prompt to explicitly request 'short, distinct, specific' queries and remove numbering in post-processing.

11. Key Learnings

11.1 Technical Skills Gained

- Setting up and operating a local LLM runtime using LM Studio, including model selection, server configuration, and API integration.
- Using LangChain's ChatOpenAI interface to communicate with any OpenAI-compatible API endpoint, whether local or cloud-hosted.
- Building multi-agent workflows using LangGraph's StateGraph, including node definition, edge configuration, and state schema design with TypedDict.
- Implementing LangGraph's MemorySaver for persistent, thread-keyed session memory in a stateful application.
- Integrating the Tavily API for structured web search retrieval optimized for LLM consumption.
- Building a Streamlit application with custom HTML/CSS styling, session state management, and form-based input handling.
- Using python-dotenv to manage environment variables securely, preventing sensitive credentials from being committed to version control.
- Designing a modular Python package with clean interfaces between components (opensearch/ package with Planner, Searcher, Writer classes).

11.2 Architectural Insights

- The Single Responsibility Principle significantly improves debuggability — when the Planner generates poor queries, the fix is isolated to planner.py without touching the rest of the system.
- Separating search from summarization (Searcher returns raw content, Writer synthesizes) produces better output quality because the LLM has more context for grounded reasoning.
- LangGraph's thread_id-based memory model maps naturally to a multi-session chat application, making it a good architectural fit for this type of project.
- Prompt engineering is as important as code quality — the same system with different prompts can produce dramatically different output quality.

11.3 Project Management Learnings

- Communicating hardware limitations to the mentor early allowed for a smooth model switch without disrupting the project timeline.
- Validating each milestone's core functionality before moving to the next prevented compounding bugs.
- Keeping a bug log helped track what had been tried and prevented re-investigating the same root causes.

12. Future Improvements

12.1 Short-Term

- Add a PDF export option using fpdf2 or weasyprint so reports can be saved and shared as formatted PDFs rather than Markdown files.
- Implement a reflection loop where the Writer Agent evaluates its own summary quality and requests additional Searcher queries if information gaps are detected.
- Add a confidence score or source quality rating to each research result to help users assess the reliability of the output.
- Support file attachment — allow users to upload PDFs or text files as additional context for research queries.

12.2 Medium-Term

- Deploy the application to a cloud platform (e.g., Streamlit Community Cloud or a VPS) so it can be accessed from any device without a local setup.
- Add support for multiple LLM backends (Ollama, OpenAI API, Groq) configurable from the UI without code changes.
- Implement multi-step research with automated follow-up — the agent autonomously generates and answers sub-questions until a confidence threshold is met.
- Add user authentication and per-user thread isolation for multi-user deployment scenarios.

12.3 Long-Term

- Build a knowledge graph layer that connects research findings across multiple sessions, enabling cross-topic insights and trend identification.
- Integrate academic database APIs (arXiv, PubMed, Semantic Scholar) alongside Tavily for domain-specific, peer-reviewed source retrieval.
- Develop a browser extension that can trigger research queries directly from web pages the user is reading.

13. Conclusion

OpenDeepResearcher successfully demonstrates the power of combining local LLMs, agentic orchestration, and real-time web search to create a functional, autonomous research assistant. The project progressed through all four milestones on schedule, building from a basic environment setup to a fully functional web application with session memory, structured report generation, and downloadable outputs.

The modular multi-agent architecture proved to be a sound design choice. Each agent's single responsibility made the system easy to debug, extend, and improve. The integration of LangGraph's MemorySaver with thread-based memory provides genuine conversational continuity that significantly improves the user experience compared to a stateless query-response system.

The key technical challenges — hardware limitations, API configuration bugs, and Streamlit state management — were each resolved systematically, and the lessons learned from each challenge strengthened both the final implementation and the developer's understanding of the underlying technologies.

The project achieves its core objective: a user can enter any research topic, and the system autonomously plans, searches, synthesizes, and delivers a comprehensive, citable research report in seconds — a task that would take a human researcher hours to complete manually.

References

- LangGraph Documentation — https://python.langchain.com/docs/langgraph
- LangChain Documentation — https://python.langchain.com/docs
- LM Studio — https://lmstudio.ai
- Tavily API Documentation — https://docs.tavily.com
- Streamlit Documentation — https://docs.streamlit.io
- Qwen2.5 Model Card — https://huggingface.co/Qwen/Qwen2.5-3B-Instruct
- Python python-dotenv — https://pypi.org/project/python-dotenv/